

L06 – Graph Neural Networks (GNN)

Foundations Lecture Series

Course Instructor

Microgrid 3-phase Security AI Project

Foundations Lecture 6

Keywords: message passing, GCN, GraphSAGE, GAT, graph readout, oversmoothing

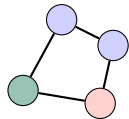
Lecture roadmap

- 1 Where graphs show up and why MLPs are not enough
- 2 Message passing: the one core idea
- 3 Major variants: GCN, GraphSAGE, GAT
- 4 Three task families: node, edge, graph level
- 5 Worked example: a 4-bus feeder as a graph
- 6 Known limitations: oversmoothing, expressivity, small data
- 7 How the project's Week 7 GNN is built

Prerequisite

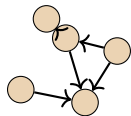
L05 (MLPs and PyTorch). Helpful: a vague memory of adjacency matrices and basic graph terminology (nodes, edges, neighbours).

Graph-structured data is everywhere



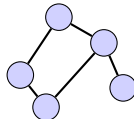
Molecule

atoms / bonds



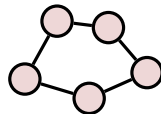
Citation network

papers / citations



Transport network

stations / roads



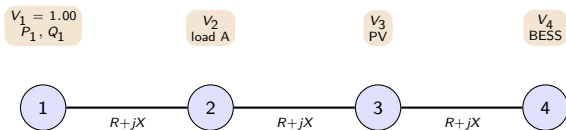
Power grid

buses / lines

The common thread

In each example the data has two kinds of entity: **nodes** (things) and **edges** (relationships). The interesting signal depends on *which* nodes are connected, not just their individual features.

A microgrid is a graph



Graph elements

- **Nodes** = buses. Each has features: V , P , Q , load and DER profile, scenario metadata.
- **Edges** = lines / transformers. Each has features: R , X , rating, on/off mask.
- **Graph-level** features: scenario type, time of day, contingency descriptor.

Targets

- Node-level: per-bus voltage.
- Edge-level: per-line loading.
- **Graph-level**: “will this state violate after any contingency?” *This is our project’s task.*

Why an MLP on flattened features falls short

Naive approach: take every bus's features, stack into one long vector, feed to an MLP.

$$\mathbf{x}_{\text{flat}} = [h_1 \mid h_2 \mid \dots \mid h_N] \in \mathbb{R}^{N \cdot d}, \quad \hat{y} = \text{MLP}(\mathbf{x}_{\text{flat}})$$

What the MLP cannot do

- It does not know that bus 3 is adjacent to bus 4 but not bus 1.
- It cannot tell a 5-bus feeder from a 10-bus feeder (input size differs).
- It must *relearn* the relationship between adjacent buses from data.
- Heavy permutation sensitivity: shuffling the bus order gives a different prediction.

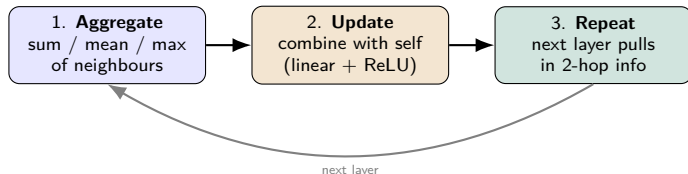
What we want

A model that takes a *graph* as input directly, respects the topology, and works on any size / shape of network. That is a Graph Neural Network.

Project's honest baseline

On 77 samples, a flat-MLP often *wins* simply because there isn't enough data to use the topology. We still build the GNN – it's the right framework for scaling up.

Message passing in three steps



In words. At each layer:

- 1 Each node *collects* feature vectors from its neighbours.
- 2 It *combines* them with its own current state via a small learned function (typically: linear layer + nonlinearity).
- 3 After L layers, every node has aggregated information from its L -hop neighbourhood.

The key insight

A GNN is just an MLP with a particular weight-tying pattern: *the same small network is applied at every node, using its local neighbourhood as input*. Topology comes in through the neighbour list.

The general message-passing equation

For each node v at layer l :

$$h_v^{(l+1)} = \text{UPDATE}\left(h_v^{(l)}, \text{AGG}\left(\{h_u^{(l)} : u \in \mathcal{N}(v)\}\right)\right)$$

Common choices

- **AGG**: sum, mean, max, attention-weighted sum.
- **UPDATE**: linear + ReLU, GRU cell, MLP.

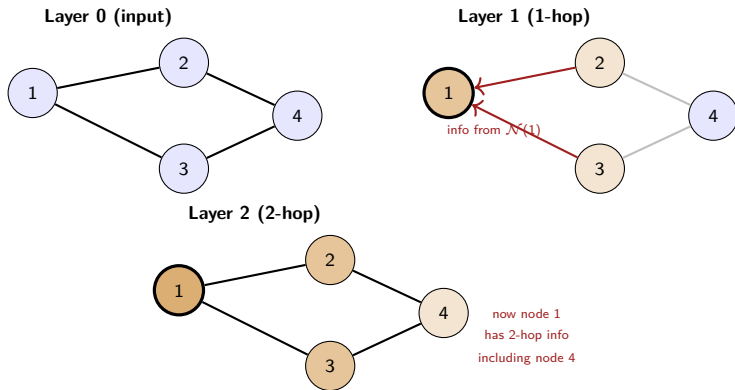
Important properties

- AGG must be *permutation-invariant* (no “first neighbour”).
- All nodes share the same weights at a given layer (parameter sharing).

Different choices = different GNNs

The next three slides each pick a specific AGG and UPDATE: GCN, GraphSAGE, and GAT. They differ in expressivity and in how they handle different neighbourhood sizes.

Visualizing message passing on a 4-node graph



Receptive field

After L layers, node v 's embedding contains information about all nodes within L hops. For our 4-bus radial example, 2 or 3 layers is enough.

GCN – Graph Convolutional Network (Kipf & Welling, 2017)

The cleanest baseline. Layer-wise update:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)}\right)$$

where

- $\tilde{A} = A + I$ is the adjacency matrix with self-loops (so each node aggregates itself too).
- $\tilde{D}$ is the diagonal degree matrix of $\tilde{A}$.
- $W^{(l)}$ is a learnable weight matrix (the same MLP layer applied at every node).

Per-node view

$$h_v^{(l+1)} = \sigma\left(W^{(l)} \cdot \sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{\sqrt{\deg(v) + 1} \sqrt{\deg(u) + 1}} h_u^{(l)}\right)$$

A weighted mean of self and neighbours, then linear projection, then nonlinearity. The normalization keeps activations stable.

GraphSAGE – learnable aggregator (Hamilton et al., 2017)

$$h_v^{(l+1)} = \sigma(W^{(l)} \cdot [h_v^{(l)}, \text{AGG}(\{h_u^{(l)} : u \in \mathcal{N}(v)\})])$$

Key differences from GCN:

- Concatenate self and aggregated neighbours (not sum them).
- AGG is one of: mean, max-pool, sum, LSTM.
- No global normalization. Easier to apply to graphs of varying size.

Inductive

- Trained on one graph, can be applied to *unseen* graphs.
- Useful when test feeders differ from training feeders.

Practical

- Mean or max-pool aggregator is fast and competitive.
- LSTM aggregator is more expressive but slower.

GAT – Graph Attention Network (Veličković et al., 2018)

$$h_v^{(l+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \alpha_{vu} W^{(l)} h_u^{(l)}\right)$$

where α_{vu} is a *learned attention coefficient*:

$$\alpha_{vu} = \text{softmax}_u(\text{LeakyReLU}(a^\top [Wh_v \parallel Wh_u]))$$

What this buys

- Each neighbour gets a learned importance weight.
- Multi-head attention (run K in parallel, concatenate) increases expressivity.
- Works well when some neighbours matter much more than others (e.g., a heavily-loaded line).

Cost

More parameters, more computation, more risk of overfitting on small graphs. Use when GCN / SAGE underperform and you have the data to support it.

Picking a variant: rough guide

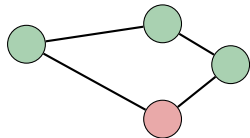
	GCN	GraphSAGE	GAT	Plain MLP
Topology-aware	✓	✓	✓	—
Inductive use on unseen graphs	possible with shared features	designed for it	possible with shared features	only with fixed-size encoding
Edge features used naturally	limited	limited	limited	—
Parameter count	smallest	small	medium	smallest
Small-data behavior	can overfit	can overfit	highest overfit risk	strong baseline
Scaling behavior	full-batch or sampled	neighbor sampling is native	attention is costlier	ignores explicit topology

For the project

Week 7 uses a GCN-style backbone, customised with per-phase bus channels and a contingency mask. Plain MLP is the comparison baseline. With only 77 samples, architecture ranking is an empirical question rather than a rule inferred from model names.

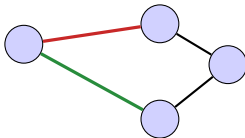
Node-level, edge-level, graph-level

Node-level



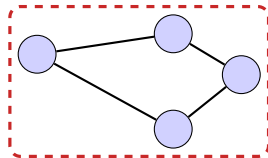
predict per-bus voltage

Edge-level



predict per-line loading

Graph-level (project)



predict "violation" label

All three use the same backbone

The GNN computes node embeddings. The *readout head* on top determines the task: per-node MLP, per-edge MLP, or a graph pooling layer followed by an MLP. We will use the third for the project.

Pooling to graph-level

After L layers we have an embedding h_v at every node. To produce a single graph-level prediction:

$$\mathbf{z}_G = \text{POOL}(\{h_v : v \in V\}), \quad \hat{y} = \text{MLP}(\mathbf{z}_G)$$

Common POOL choices:

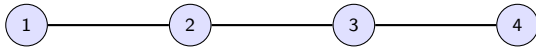
- **mean pool:** $\mathbf{z}_G = \frac{1}{|V|} \sum_v h_v$ – bias toward typical bus state.
- **max pool:** $\mathbf{z}_G = \max_v h_v$ (elementwise) – bias toward worst-case bus.
- **sum pool:** $\mathbf{z}_G = \sum_v h_v$ – preserves overall magnitude.
- **attention pool:** learned weights α_v over nodes.

For safety screening, max-pool is a sensible default

The graph is unsafe *if any one bus* crosses a limit. Max-pool naturally captures this “worst case wins” logic. Mean-pool can hide one bad bus by averaging it with many good ones.

Setup: a feeder, its features, and the task

$$h_1 = (1.00, 0.0, 0.0) \quad h_2 = (0.98, 0.10, 0.04) \quad h_3 = (0.97, 0.15, 0.06) \quad h_4 = (0.95, 0.20, 0.08)$$



- Each node has feature $h_v = (V_v, P_v, Q_v)$ – voltage and net injection.
- Edges have features (R, X) – omitted for the cleanest derivation.
- Task: predict a single binary label “violation after any contingency”.

Strategy

2 GCN layers $\rightarrow$ max-pool node embeddings $\rightarrow$ MLP head $\rightarrow$ sigmoid. Train with BCE on labeled scenarios.

One GCN layer, by hand

Using the simplified mean aggregator (close enough to GCN for intuition):

$$h_v^{(1)} = \sigma \left(W \cdot \frac{1}{|\mathcal{N}(v)| + 1} \sum_{u \in \mathcal{N}(v) \cup \{v\}} h_u^{(0)} \right)$$

For node 2, $\mathcal{N}(2) = \{1, 3\}$. The aggregated input (mean of itself, node 1, node 3) is:

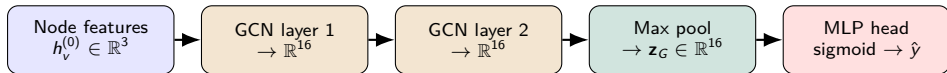
$$\bar{h}_2 = \frac{1}{3}((0.98, 0.10, 0.04) + (1.00, 0.0, 0.0) + (0.97, 0.15, 0.06)) = (0.983, 0.083, 0.033)$$

The learnable matrix W then maps this 3-vector to a hidden representation (e.g., 16-dim), and $\sigma = \text{ReLU}$.

What just happened

Node 2's new embedding $h_2^{(1)}$ depends on its neighbours' features. After one layer it knows about nodes 1 and 3 (1-hop neighbourhood). After two layers it will also know about node 4 (via node 3).

After two layers + pooling, we get a risk score



End-to-end forward pass

- Node features in $\mathbb{R}^{N \times 3}$.
- After 2 GCN layers: $\mathbb{R}^{N \times 16}$ embeddings.
- After max-pool over nodes: a single $\mathbb{R}^{16}$ graph vector.
- MLP head with sigmoid: scalar risk score $\hat{y} \in [0, 1]$.

Training

Same as L05: minibatch of (graph, label) pairs, BCE loss, Adam, early stop on validation. Use class-weighted BCE for the project's 47/30 imbalance.

In code: plain PyTorch vs PyTorch Geometric

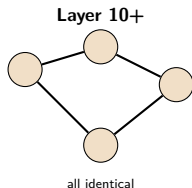
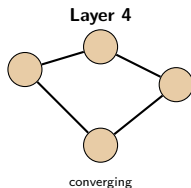
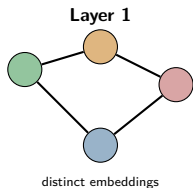
```
# Option A: PyTorch Geometric (production standard)
import torch.nn as nn
from torch_geometric.nn import GCNConv, global_max_pool

class GridGNN(nn.Module):
    def __init__(self, in_dim=3, hidden=16):
        super().__init__()
        self.g1 = GCNConv(in_dim, hidden)
        self.g2 = GCNConv(hidden, hidden)
        self.head = nn.Sequential(nn.Linear(hidden, hidden), nn.ReLU(),
                                   nn.Linear(hidden, 1), nn.Sigmoid())
    def forward(self, x, edge_index, batch):
        h = self.g1(x, edge_index).relu()
        h = self.g2(h, edge_index).relu()
        z = global_max_pool(h, batch) # batch indicates which graph each node belongs to
        return self.head(z)
```

What the project actually uses

The Week 7 notebook implements the GCN-style message passing in *plain PyTorch* (no PyG dependency) so students can read every line. Same math, more lines of code, no extra install.

Oversmoothing



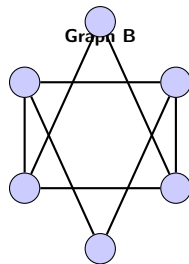
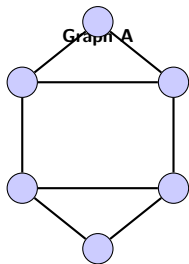
What's happening

Each layer mixes a node with its neighbours. After many layers, every node's embedding becomes a global average – node identity is washed out.

Practical advice

Limit depth to 2–4 layers for most problems. If you need deeper, use residual connections, jumping knowledge, or DropEdge.

Expressivity bounds (Weisfeiler-Lehman)



The take-away

Message-passing GNNs are upper-bounded by the 1-WL graph isomorphism test, and particular aggregators can be less expressive. There exist non-isomorphic graphs they *cannot* distinguish. Structural features or higher-order architectures can go beyond this limit.

Small-data behavior: an honest finding from the project

Holdout model (Week 7 of the project)	Recall	PF calls saved
GradientBoosting (ML baseline)	1.000	0.400
PI-MLP (full physics loss)	1.000	0.400
FlattenedMLP (topology-blind)	1.000	0.400
PhaseAwareGNN	0.583	0.600

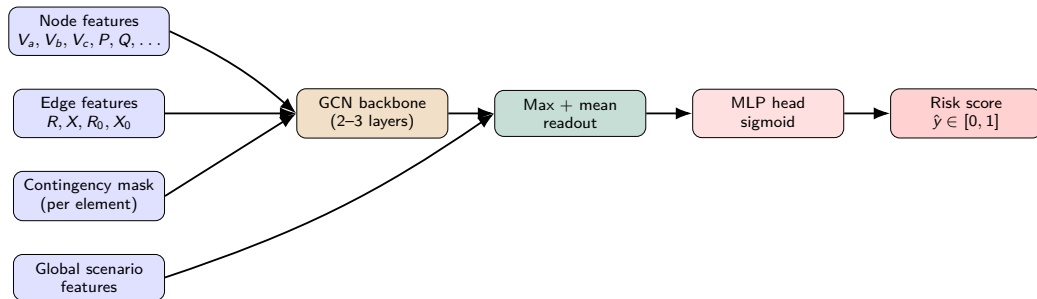
Why GNN loses here

- Only 77 labeled scenarios.
- GNN parameter sharing helps when graph diversity is high – but we have one feeder.
- Flat MLP can memorize 77 patterns; GNN has stronger inductive bias that doesn't pay off at this scale.

We still build the GNN

The architecture is right; the dataset is too small. Scale to hundreds of feeders and the picture inverts. This is a real lesson about how AI choices depend on data regime, not just method elegance.

Week 7's phase-aware GNN at a high level



What is “phase-aware”

Each bus carries *three* voltage channels (V_a, V_b, V_c) instead of one balanced positive-sequence voltage. The same node-feature vector contains phase-A, B, C quantities together, so the GNN learns from the unbalance pattern directly.

Reading the project notebook

With L05 (PyTorch) + L06 (message passing) in hand, the Week 7 notebook is now within reach. Don't skip the validation cells – they verify graph construction, mask correctness, and per-phase consistency.

Homework

Required

- 1 Train a 2-layer GCN on the Cora citation dataset using PyTorch Geometric (or plain PyTorch). Achieve $\geq 80\%$ test accuracy on node classification.
- 2 Visualize the learned node embeddings with t-SNE (or PCA) at layer 1 and layer 2. Are the classes separating?
- 3 Replace GCN with GraphSAGE (mean aggregator). Compare test accuracy. Run each with 3 seeds and report mean / stdev.
- 4 Implement a tiny graph-level classifier: 4-bus feeder graphs labeled by your own rule (e.g. unsafe if $V_{\min} < 0.95$). Train a 2-layer GCN with max-pool readout. Report recall on a held-out set.

Optional

- 1 Increase the GCN depth from 2 to 8 layers. Plot validation accuracy vs depth. Where does oversmoothing kick in?
- 2 Try edge features by manually concatenating (R, X) to each message. Does it help?

Exit checklist

- ① ✓ I can explain in one sentence why an MLP on flattened features misses graph structure.
- ② ✓ I can write the general message-passing equation and identify the aggregator and updater.
- ③ ✓ I can describe how GCN, GraphSAGE, and GAT differ in one sentence each.
- ④ ✓ I can match the three task levels (node, edge, graph) to power-grid examples.
- ⑤ ✓ I can sketch the data flow from node features through 2 GCN layers and a pooling layer to a risk score.
- ⑥ ✓ I know what oversmoothing is and why $\text{depth} > 4$ is usually a bad idea on small graphs.
- ⑦ ✓ I can read the Week 7 phase-aware GNN code without consulting external docs.

References

- **Hamilton**, *Graph Representation Learning* (free book, 2020). https://www.cs.mcgill.ca/~wlh/grl_book/. Chapters 1–5 cover everything in this lecture.
- **Kipf & Welling**, 2017. *Semi-Supervised Classification with Graph Convolutional Networks*, ICLR. The GCN paper.
- **Hamilton, Ying, Leskovec**, 2017. *Inductive Representation Learning on Large Graphs*, NeurIPS. GraphSAGE.
- **Veličković et al.**, 2018. *Graph Attention Networks*, ICLR. GAT.
- **Xu et al.**, 2019. *How Powerful Are Graph Neural Networks?*, ICLR. WL bounds and GIN.
- **PyTorch Geometric documentation**: <https://pytorch-geometric.readthedocs.io/>.
- **Donon et al.**, 2020. *Graph Neural Solver for Power Systems*, IJCNN. Power-grid GNN application.